

CODING CHALLENGE-CO3-AT1

SUTDENT NAME: P. VISHNU VARDHAN REDDY

REG. NO: 192425212

COURSE CODE: MLA0302

COURSE NAME: REINFORCEMENT LEARNING

1. Predict the Reinforcement Learning (RL) framework by describing the agent–environment interaction, emphasizing the importance of states, actions, and rewards, and discuss how cumulative reward influences decision-making.

Aim

To understand the Reinforcement Learning framework by implementing agent–environment interaction using states, actions, and rewards.

Python Code

```
import random

states = ["Start", "Middle", "Goal"]
actions = ["Move", "Stay"]

state = "Start"
total_reward = 0
gamma = 0.9

for step in range(10):

    # Agent selects an action
    action = random.choice(actions)

    # Environment response
    if state == "Start" and action == "Move":
        next_state = "Middle"
        reward = 1

    elif state == "Middle" and action == "Move":
        next_state = "Goal"
        reward = 10

    elif state == "Goal":
```

```
    next_state = "Goal"
    reward = 0

else:
    next_state = state
    reward = -1

print("State:", state,
      "Action:", action,
      "Reward:", reward)

total_reward += reward
state = next_state

if state == "Goal":
    break

print("Total Reward:", total_reward)
```

Output

```
... State: Start Action: Stay Reward: -1
    State: Start Action: Stay Reward: -1
    State: Start Action: Move Reward: 1
    State: Middle Action: Move Reward: 10
    Total Reward: 9
```

Conclusion

The agent interacts with the environment through states, actions, and rewards. Cumulative reward helps the agent select actions that provide better long-term results.

2. Estimate how decision-making problems can be modeled as a Markov Decision Process (MDP), detailing the components of action space, transition probability, reward function, and discount factor.

Aim

To model a decision-making problem as a Markov Decision Process using states, actions, transition probabilities, rewards, and discount factor.

Python Code

```

states = ["Low", "Medium", "High"]

actions = ["Short", "Medium", "Long"]

gamma = 0.9

rewards = {
    ("Low", "Short"): 5,
    ("Low", "Medium"): 3,
    ("Low", "Long"): 1,

    ("Medium", "Short"): 2,
    ("Medium", "Medium"): 5,
    ("Medium", "Long"): 3,

    ("High", "Short"): -2,
    ("High", "Medium"): 3,
    ("High", "Long"): 6
}

transition_probability = {
    ("Low", "Short"): {"Low": 0.8, "Medium": 0.2},
    ("Medium", "Medium"): {"Medium": 0.7, "High": 0.3},
    ("High", "Long"): {"High": 0.8, "Medium": 0.2}
}

print("States:", states)
print("Actions:", actions)
print("Discount Factor:", gamma)

print("\nRewards:")
for key, value in rewards.items():
    print(key, "=", value)

print("\nTransition Probabilities:")
for key, value in transition_probability.items():
    print(key, "->", value)

```

Output

```
... States: ['Low', 'Medium', 'High']
    Actions: ['Short', 'Medium', 'Long']
    Discount Factor: 0.9

    Rewards:
    ('Low', 'Short') = 5
    ('Low', 'Medium') = 3
    ('Low', 'Long') = 1
    ('Medium', 'Short') = 2
    ('Medium', 'Medium') = 5
    ('Medium', 'Long') = 3
    ('High', 'Short') = -2
    ('High', 'Medium') = 3
    ('High', 'Long') = 6

    Transition Probabilities:
    ('Low', 'Short') -> {'Low': 0.8, 'Medium': 0.2}
    ('Medium', 'Medium') -> {'Medium': 0.7, 'High': 0.3}
    ('High', 'Long') -> {'High': 0.8, 'Medium': 0.2}
```

Conclusion

The MDP provides a mathematical representation of sequential decision-making under uncertainty using states, actions, transitions, rewards, and discounting.

3. Discuss the role of policy and value functions in RL, and examine how state-value and action-value functions contribute to evaluating long-term benefits of actions using the Bellman Equation.

Aim

To understand policy, state-value, and action-value functions using the Bellman Equation.

Python Code

```
states = ["A", "B", "Goal"]
```

```
actions = {
```

```

    "A": ["Right", "Down"],
    "B": ["Right"],
    "Goal": []
}

rewards = {
    ("A", "Right"): 2,
    ("A", "Down"): 1,
    ("B", "Right"): 10
}

gamma = 0.9

V = {
    "A": 0,
    "B": 0,
    "Goal": 0
}

# Bellman value update
V["B"] = rewards[("B", "Right")] + gamma * V["Goal"]

V["A"] = max(
    rewards[("A", "Right")] + gamma * V["B"],
    rewards[("A", "Down")] + gamma * V["B"]
)

print("State Values:")
for state, value in V.items():
    print(state, "=", value)

Q_A_Right = rewards[("A", "Right")] + gamma * V["B"]
Q_A_Down = rewards[("A", "Down")] + gamma * V["B"]

print("\nQ(A, Right) =", Q_A_Right)
print("Q(A, Down) =", Q_A_Down)

```

Output

```

print(Q(A, Down) = , Q_

... State Values:
    A = 11.0
    B = 10.0
    Goal = 0

    Q(A, Right) = 11.0
    Q(A, Down) = 10.0

```

Conclusion

Value functions evaluate the long-term usefulness of states and actions. The Bellman Equation recursively combines immediate and future rewards to calculate these values.

4. Justify the exploration–exploitation trade-off in RL and compare strategies such as ϵ -greedy and softmax approaches in balancing learning efficiency.

Aim

To implement and compare ϵ -greedy and Softmax strategies for balancing exploration and exploitation.

Python Code

```
import random
import numpy as np

q_values = [2, 5, 3]
actions = ["A", "B", "C"]

epsilon = 0.1

# Epsilon-Greedy
if random.random() < epsilon:
    selected = random.choice(actions)
    print("Exploration")
else:
    selected = actions[np.argmax(q_values)]
    print("Exploitation")

print("Epsilon-Greedy Action:", selected)

# Softmax
temperature = 1.0

exp_values = np.exp(np.array(q_values) / temperature)
probabilities = exp_values / np.sum(exp_values)

selected_softmax = np.random.choice(
    actions,
    p=probabilities
)

print("\nSoftmax Probabilities:", probabilities)
print("Softmax Action:", selected_softmax)
```

Output

```
... Exploitation
    Epsilon-Greedy Action: B

    Softmax Probabilities: [0.04201007 0.84379473 0.1141952 ]
    Softmax Action: B
```

Conclusion

ϵ -greedy provides simple random exploration, whereas Softmax assigns probabilities based on Q-values. Both strategies help the agent balance exploration and exploitation.

5. Explain how reward shaping can accelerate learning in RL and illustrate its impact on preventing suboptimal policies.

Aim

To demonstrate how reward shaping provides intermediate feedback and helps an RL agent learn a better path toward a goal.

Python Code

```
position = 0
goal = 5
total_reward = 0

for step in range(10):

    old_distance = abs(goal - position)

    # Move right
    new_position = position + 1

    new_distance = abs(goal - new_position)

    if new_position == goal:
        reward = 10
    elif new_distance < old_distance:
        reward = 2
    else:
        reward = -1

    total_reward += reward
    position = new_position
```

```
print("Position:", position,
      "Reward:", reward)

if position == goal:
    break

print("Total Shaped Reward:", total_reward)
```

Output

```
...   Position: 1 Reward: 2
      Position: 2 Reward: 2
      Position: 3 Reward: 2
      Position: 4 Reward: 2
      Position: 5 Reward: 10
      Total Shaped Reward: 18
```

Conclusion

Reward shaping provides intermediate feedback and can accelerate learning by guiding the agent toward desirable states. Carefully designed rewards can also reduce the possibility of learning suboptimal behavior.

6. Identify the challenges of applying RL in real-world robotics and analyze how safety, sample efficiency, and environment variability affect agent performance.

Aim

To demonstrate important challenges in applying Reinforcement Learning to a real-world robotic environment, particularly safety, sample efficiency, and environmental variability.

Python Code

```
import random

position = 0
goal = 5
total_reward = 0
```

```
for step in range(10):

    action = random.choice([-1, 1])

    new_position = position + action

    # Safety constraint
    if new_position < 0 or new_position > goal:
        reward = -10
        print("Unsafe action!")
        continue

    # Environment variability
    noise = random.choice([-1, 0, 1])
    actual_position = new_position + noise

    if actual_position == goal:
        reward = 10
    else:
        reward = -1

    total_reward += reward
    position = max(0, min(goal, actual_position))

    print("Position:", position,
          "Action:", action,
          "Reward:", reward)

    if position == goal:
        break

print("Total Reward:", total_reward)
```

Output

```
... Unsafe action!
    Position: 0 Action: 1 Reward: -1
    Position: 2 Action: 1 Reward: -1
    Position: 4 Action: 1 Reward: -1
    Position: 5 Action: 1 Reward: -1
    Total Reward: -4
```

Conclusion

Real-world RL faces safety risks, expensive training samples, sensor noise, and changing environments. Safety constraints and simulation can help improve performance before physical deployment.

7. Analyze the influence of the discount factor (γ) on short-term versus long-term decision-making, and evaluate its role in episodic tasks.

Aim

To demonstrate the effect of the discount factor γ on short-term and long-term decision-making.

Python Code

```
rewards = [5, 10, 20]

for gamma in [0.1, 0.5, 0.9]:

    return_value = (
        rewards[0]
        + gamma * rewards[1]
        + gamma**2 * rewards[2]
    )

    print("Gamma =", gamma)
    print("Cumulative Return =", return_value)
    print()
```

Output

```
*** Gamma = 0.1
    Cumulative Return = 6.2

    Gamma = 0.5
    Cumulative Return = 15.0

    Gamma = 0.9
    Cumulative Return = 30.200000000000003
```

Conclusion

The discount factor controls the balance between short-term and long-term rewards. A high γ is useful when long-term outcomes are important, especially in multi-step episodic tasks.

8. Evaluate the effectiveness of model-free RL compared to model-based RL in dynamic environments, and differentiate their strengths and limitations.

Aim

To compare model-free and model-based Reinforcement Learning approaches in terms of learning and decision-making.

Python Code

```
# Simple environment

states = [0, 1, 2]
actions = ["Left", "Right"]

# Model-based: known transition model
transition_model = {
    (0, "Right"): 1,
    (1, "Right"): 2,
    (1, "Left"): 0,
    (2, "Left"): 1
}

rewards = {
    (0, "Right"): 1,
    (1, "Right"): 10,
    (1, "Left"): -1,
    (2, "Left"): -1
}

print("MODEL-BASED RL")
print("-----")

state = 0

for action in ["Right", "Right"]:

    next_state = transition_model[(state, action)]
    reward = rewards[(state, action)]

    print(
        "State:", state,
        "Action:", action,
        "Next State:", next_state,
```

```

        "Reward:", reward
    )

    state = next_state

# Model-free Q-values
print("\nMODEL-FREE RL")
print("-----")

Q = {}

for state in states:
    for action in actions:
        Q[(state, action)] = 0

# Learn directly from experience
Q[(0, "Right")] = 1
Q[(1, "Right")] = 10

print("Learned Q-values:")
for key, value in Q.items():
    print(key, "=", value)

```

Output

```

...  MODEL-BASED RL
-----
State: 0 Action: Right Next State: 1 Reward: 1
State: 1 Action: Right Next State: 2 Reward: 10

MODEL-FREE RL
-----
Learned Q-values:
(0, 'Left') = 0
(0, 'Right') = 1
(1, 'Left') = 0
(1, 'Right') = 10
(2, 'Left') = 0
(2, 'Right') = 0

```

Conclusion

Model-free RL learns directly from experience without explicitly modeling the environment. **Model-based RL** uses an environment model for prediction and planning. Model-free methods are simpler, while model-based methods can be more sample-efficient.

9. Illustrate with an example how Q-learning updates the action-value function during training, and demonstrate its convergence behavior.

Aim

To implement the Q-learning algorithm and demonstrate how action-value functions are updated during training.

Python Code

```
import numpy as np
import random

states = 5
actions = 2

# 0 = Left, 1 = Right
Q = np.zeros((states, actions))

alpha = 0.5
gamma = 0.9
epsilon = 0.1

episodes = 100

for episode in range(episodes):

    state = 0

    while state != 4:

        # Epsilon-greedy action selection
        if random.random() < epsilon:
            action = random.randint(0, 1)
        else:
            action = np.argmax(Q[state])

        # Environment
        if action == 1:    # Right
            next_state = min(state + 1, 4)
        else:             # Left
            next_state = max(state - 1, 0)

        if next_state == 4:
            reward = 10
        else:
            reward = -1

        # Q-learning update
        Q[state, action] = Q[state, action] + alpha * (
            reward
```

```

        + gamma * np.max(Q[next_state])
        - Q[state, action]
    )

    state = next_state

print("Final Q-Table:")
print(Q)

print("\nBest Actions:")
for state in range(4):
    print(
        "State", state,
        "-> Action",
        np.argmax(Q[state])
    )

```

Output

```

... Final Q-Table:
[[ 2.80269326  4.58          ]
 [ 2.85362098  6.2          ]
 [ 4.53855458  8.           ]
 [ 6.17559741 10.           ]
 [ 0.           0.           ]]

Best Actions:
State 0 -> Action 1
State 1 -> Action 1
State 2 -> Action 1
State 3 -> Action 1

```

Conclusion

Q-learning learns the optimal action-value function through repeated experience. With sufficient training, the Q-values converge toward useful estimates of the optimal values.

10 Differentiate between on-policy and off-policy learning methods in RL, and assess their advantages and limitations in practical applications.

Aim

To understand and compare on-policy and off-policy learning using SARSA and Q-learning.

Theory

Python Code

```
import numpy as np

alpha = 0.5
gamma = 0.9

# Initial Q-values
Q_sarsa = np.zeros((3, 2))
Q_qlearning = np.zeros((3, 2))

state = 0
action = 1

next_state = 1
next_action = 0
reward = 5

# -----
# SARSA - On Policy
# -----

Q_sarsa[state, action] = Q_sarsa[state, action] + alpha * (
    reward
    + gamma * Q_sarsa[next_state, next_action]
    - Q_sarsa[state, action]
)

# -----
# Q-Learning - Off Policy
# -----

Q_qlearning[state, action] = Q_qlearning[state, action] + alpha * (
    reward
    + gamma * np.max(Q_qlearning[next_state])
    - Q_qlearning[state, action]
)

print("SARSA Q-Table:")
print(Q_sarsa)

print("\nQ-Learning Q-Table:")
print(Q_qlearning)
```

Output

```
... SARSA Q-Table:
[[0.  2.5]
 [0.  0. ]
 [0.  0. ]]

Q-Learning Q-Table:
[[0.  2.5]
 [0.  0. ]
 [0.  0. ]]
```

Conclusion

SARSA learns using the action actually selected by the current policy, while Q-learning learns using the best possible next action. SARSA can be useful when actual behavior and safety matter, whereas Q-learning is effective when the objective is to learn an optimal policy.